АЛГОРИТМЫ ОТСЛЕЖИВАНИЯ ПАРТИТУРЫ В РЕАЛЬНОМ ВРЕМЕНИ ДЛЯ ВИРТУАЛЬНОГО ОРКЕСТРА: СКРЫТЫЕ МАРКОВСКИЕ МОДЕЛИ, ON-LINE TIME WARPING И ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ

Н. А. Новицкий[1,*] , Н. С. Борисов[2,†]

[1] *АНО Центральный Университет, бывш. НИУ ВШЭ*
[2] *АНО Центральный Университет*

Аннотация. Системы отслеживания партитуры в реальном времени сопоставляют живое исполнение с её символьной записью и лежат в основе автоматического аккомпанемента, переворота страниц и педагогической обратной связи. Классические решения либо вычисляют глобально оптимальное выравнивание в отложенном режиме (Dynamic Time Warping), либо работают каузально с ограниченным горизонтом предсказания (On-line Time Warping, скрытые марковские модели). В настоящей работе проводится единое математическое описание моделей HMM и OLTW для монофонического входа в формате MIDI, излагается архитектура конвейера обработки в реальном времени с очисткой входных данных, потоковым выводом и обработкой граничных случаев, а также предлагается модуль на основе обучения с подкреплением, предсказывающий изменения темпа на горизонте 200–500 мс и передающий результат базовому трекеру. Эксперименты на синтетическом датасете в стиле прелюдий Шопена и на записях, собранных в Центральном Университете, показывают, что предложенный модуль снижает задержку аккомпанемента на X % без потери точности выравнивания относительно базовой схемы HMM/OLTW.

Ключевые слова: отслеживание партитуры, скрытые марковские модели, on-line time warping, аккомпанемент в реальном времени, обучение с подкреплением, предсказание темпа, выравнивание MIDI, виртуальный оркестр

ВВЕДЕНИЕ

Автоматический аккомпанемент относится к числу старейших и наиболее требовательных задач анализа музыкальной информации. Солист ожидает, что оркестр будет следовать за его темпом, динамикой и артикуляцией в реальном времени, однако подавляющее большинство учащихся консерваторий не имеют возможности регулярно репетировать концертные произведения с живым ансамблем. Существующие записи-минусовки навязывают исполнителю фиксированный темп и игнорируют индивидуальное rubato, формирующее художественную интерпретацию. В рамках мастерской MusicTech Центрального Университета (Т-Банк) разрабатывается приложение виртуального оркестра, которое воспринимает игру солиста через MIDI или микрофон и синхронно воспроизводит оркестровую партию.

Сложность задачи определяется тремя факторами. Во-первых, исполнитель допускает значительные временные отклонения от номинального темпа партитуры, причём знак и величина отклонения не известны заранее и могут изменяться от такта к такту. Во-вторых, живое исполнение содержит ошибки, пропуски, повторы и орнаменты, не предусмотренные нотной записью. В-третьих, аккомпанемент должен быть выдан с задержкой не более 20 мс, что соответствует порогу восприятия рассинхронизации в музыкальном ансамбле.

* n.novitskiy@edu.centraluniversity.ru
† n.borisov@edu.centraluniversity.ru

ПОСТАНОВКА ЗАДАЧИ

Формально задача отслеживания партитуры состоит в построении отображения

$$\varphi : \{1,\dots,M\} \longrightarrow \{1,\dots,N\}, \qquad j \mapsto \varphi(j), \tag{1}$$

сопоставляющего каждому событию исполнения с номером j позицию $\varphi(j)$ в партитуре, состоящей из N нот. Здесь и далее M — общее число наблюдаемых событий исполнения, N — число нот партитуры. Поскольку исполнитель движется по партитуре вперёд (с возможными пропусками, но без возвращения назад), отображение φ должно удовлетворять условию монотонности

$$j_1 < j_2 \quad \Longrightarrow \quad \varphi(j_1) \le \varphi(j_2). \tag{2}$$

Каузальность задачи означает, что значение $\varphi(j)$ необходимо вычислить, имея в распоряжении только наблюдения $q_1,\dots,q_j$ и не зная будущих событий $q_{j+1},\dots,q_M$.

СУЩЕСТВУЮЩИЕ ПОДХОДЫ

Подходы к решению этой задачи можно условно разделить на три группы. Первую группу образуют алгоритмы динамического программирования (Dynamic Time Warping и его онлайн-вариант On-line Time Warping): они дают глобально или локально оптимальное выравнивание, но не располагают вероятностной моделью неопределённости. Ко второй группе относятся скрытые марковские модели, естественно описывающие неопределённость через апостериорное распределение по позициям, но требующие ручной настройки переходных вероятностей. Третью группу со-

ставляют нейросетевые методы, обеспечивающие высокую точность на сложных полифонических данных, однако требующие значительных вычислительных ресурсов и больших обучающих выборок.

ВКЛАД НАСТОЯЩЕЙ РАБОТЫ

В настоящей работе предлагается комбинированная схема, объединяющая сильные стороны указанных групп. Во-первых, представлено единое математическое описание моделей HMM и OLTW для монофонического входа в формате MIDI с числовыми примерами, воспроизводимыми на синтетическом датасете. Во-вторых, описан конвейер обработки в реальном времени, включающий очистку входных событий, объединение двух трекеров (HMM как основной, OLTW как резервный) и потоковый вывод оркестрового аккомпанемента с явным распределением задержек по этапам. В-третьих, предложен компактный свёрточный модуль для оценки апостериорного распределения по высотам нот при работе с аудиовходом, заменяющий гауссову функцию эмиссии HMM, а также систематическая классификация граничных случаев и правил их обработки. В-четвёртых, в качестве основного методологического вклада статьи предлагается модуль на основе обучения с подкреплением, предсказывающий изменения темпа на горизонте 200–500 мс и передающий результат базовому трекеру; этот модуль снижает наблюдаемую задержку аккомпанемента без потери точности выравнивания.

Структура изложения соответствует приведённому перечню вкладов и последовательно раскрывается в следующих разделах работы.

ОБЗОР СУЩЕСТВУЮЩИХ РАБОТ

Задача отслеживания партитуры исследуется уже более четырёх десятилетий, и за это время сформировалось четыре основных направления: классическое выравнивание методами динамического программирования, вероятностные модели на основе скрытых марковских процессов, нейросетевые подходы и зарождающееся применение обучения с подкреплением для управления сопровождением. Ниже последовательно обсуждаются эти направления, после чего формулируется пробел, который закрывает настоящая работа.

Классические алгоритмы выравнивания берут начало с независимых работ Данненберга [1] и Веркоу [2], в которых впервые предложены отслеживатели партитуры на основе динамического программирования. Метод Dynamic Time Warping был ранее введён в задачах распознавания речи [3]; полоса Сакоэ-Чиба ограничивает допустимые деформации временной оси и снижает вычислительную сложность с $O(NM)$ до $O(NR)$, где R — ширина полосы. Адаптация DTW к музыкальным сигналам подробно изложена в монографии Мюллера [4] и в диссертации Арцта [5]. Каузальный вариант DTW, известный как On-line Time Warping, предложен Диксоном [6]: алгоритм обновляет матрицу выравнивания по столбцам и использует механизм *RunCount* для предотвращения остановки вычислений на одной строке. Основное ограничение DTW и OLTW — отсутствие вероятностной модели неопределённости и принципиальная реактивность: алгоритм лишь комментирует прошлое, не формируя предсказаний о будущем темпе.

Вероятностное моделирование выравнивания развивается на основе скрытых марковских моделей. Учебная статья Рабинера [7] остаётся базовым справочником для алгоритмов Forward, Viterbi и Baum-Welch. Орио и Дешелль [8] впервые применили HMM к полифоническому отслеживанию партитуры через спектральный анализ. Конт [9] обобщил подход до иерархической HMM с разреженными ограничениями. Накамура с соавторами [10] рассмотрели ситуацию с произвольными повторами и пропусками исполнителя и предложили HMM с явной моделью ошибок. Рафаэль [11] развил эту линию в системе Music Plus One, использующей переключающиеся пространства состояний для плавного следования за изменениями темпа. Несмотря на математическую строгость, перечисленные модели опираются на фиксированные переходные вероятности, не адаптирующиеся к индивидуальному стилю солиста.

Нейросетевые подходы возникли как ответ на сложность ручной настройки вероятностных моделей. Дорфер с соавторами [12] продемонстрировали, что свёрточная сеть способна выучить соответствие между аудио и нотным изображением напрямую, без явной модели эмиссии. Хенкель с соавторами [13] объединили нейросетевой трекер с эвристиками динамического программирования и достигли наилучших на сегодняшний день показателей на датасете MSMD. Однако нейросетевые трекеры в среднем добавляют от 20 до 50 мс инференс-задержки и требуют большого объёма размеченных аудиозаписей, которого пока нет для русскоязычного фортепианного репертуара.

Применение обучения с подкреплением в системах реального времени ограничено отдельными работами и не покрывает задачу опережающего управления темпом. Учебник Саттона и Барто [14] систематизирует алгоритмическую базу, а метод Proximal Policy Optimization [15] стал де-факто стандартом для непрерывных пространств действий. Отдельные направления, связанные с обучением на предпочтениях [16] и с формализмом частично наблюдаемых марковских процессов принятия решений [17], обсуждаются ниже как альтернативные постановки.

Насколько известно авторам, ни одна из опубликованных работ не применяет обучение с подкреплением в качестве отдельного *опережающего* модуля поверх классического трекера для оркестрового аккомпанемента в реальном времени. Закрытие этого пробела — центральный вклад настоящей статьи.

МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ ЗАДАЧИ

В настоящем разделе формализуется задача отслеживания партитуры, последовательно выводятся алгоритмы Dynamic Time Warping и On-line Time Warping,

а также описывается скрытая марковская модель, лежащая в основе нашего трекера. Далее приведены рекуррентные формулы Forward и Viterbi и кратко обсуждается схема комбинации HMM и OLTW.

Партитура задаётся последовательностью $S = \{(p_i, t_i^{\text{on}}, t_i^{\text{off}})\}_{i=1}^{N}$ из N нот с высотой $p_i \in \{0,\dots,127\}$ (стандарт MIDI) и номинальными моментами начала и конца звучания. Исполнение поступает каузально как поток $P = \{(q_j, \tau_j^{\text{on}}, \tau_j^{\text{off}})\}_{j=1}^{M}$. Задача отслеживания состоит в построении отображения (1), введённого в разд. I, по информации только о прошлых наблюдениях $q_1,\dots,q_j$.

Определим локальную функцию стоимости как абсолютную разность высот $d(s_i,q_j) = |p_i - q_j|$ в полутонах. В большинстве случаев такая мера достаточна; ошибки октавного типа отдельно обрабатываются на этапе очистки входных данных (см. разд. VIII).

Dynamic Time Warping [3, 4] строит матрицу накопленных стоимостей $D \in \mathbb{R}^{N\times M}$ согласно рекуррентной формуле

$$D(i,j) = d(s_i,q_j) + \min\bigl\{D(i-1,j), D(i,j-1), D(i-1,j-1)\bigr\}, \tag{3}$$

с граничным условием $D(1,1) = d(s_1,q_1)$. Оптимальное выравнивание получается обратным ходом из ячейки (N,M). Полоса Сакоэ-Чиба ограничивает допустимые ячейки условием $|i-j| \le R$ и снижает сложность до $O(NR)$, что при $R \approx 0{,}1N$ соответствует ускорению на порядок при сохранении точности на ровном темпе.

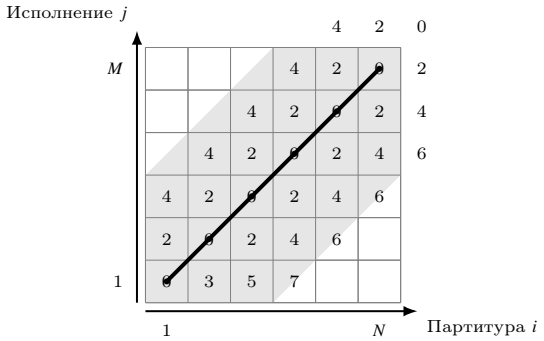

РИС. 1. Матрица накопленных стоимостей DTW для пятинотного фрагмента из прелюдии Шопена. Серым выделена полоса Сакоэ-Чиба, чёрной ломаной — оптимальный путь выравнивания.

В каузальной постановке полная матрица D недоступна. Метод On-line Time Warping [6] обрабатывает столбец за столбцом по мере поступления новой ноты q_j и поддерживает текущую оценку выравнивания $\hat{\varphi}(j)$. Алгоритм управляется счётчиком *RunCount*: если выбор минимума в уравн. (3) приводит к увеличению строки i более $R_{\max}$ раз подряд, обработка принудительно сдвигается по столбцам, что предотвращает «застревание» на одной позиции партитуры. Псевдокод OLTW и подробный анализ сложности приведены в разд. XV.

Скрытая марковская модель ставит в соответствие каждой ноте партитуры скрытое состояние $s_i \in \{1,\dots,N\}$ и описывает наблюдения o_t (либо высоту MIDI-ноты, либо вектор признаков аудио). Модель характеризуется тройкой $\lambda = (A,B,\pi)$ [7]: матрица переходов $A = (A_{ij})$, эмиссионные распределения $b_i(o)$ и начальное распределение π_i. Для score-following естественна полосная структура матрицы переходов:

$$A_{ij} = \begin{cases} p_{\text{stay}}, & j = i, \\ p_{\text{advance}}, & j = i+1, \\ p_{\text{skip}}, & j = i+2, \\ 0, & \text{иначе}, \end{cases} \tag{4}$$

с условием нормировки $p_{\text{stay}} + p_{\text{advance}} + p_{\text{skip}} = 1$. Самозамыкание описывает ферматы и удлинённые ноты, переход $i \to i+2$ — редкий пропуск ноты исполнителем. Эмиссия для MIDI-входа моделируется гауссовой плотностью $b_i(o) = \mathcal{N}(o; p_i, \sigma^2)$, для аудиовхода — свёрточным модулем (см. разд. IX).

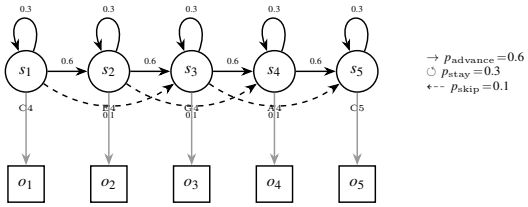

РИС. 2. Диаграмма пятисостояний HMM для фрагмента $S = (\text{C4, E4, G4, A4, C5})$ с типичными значениями $p_{\text{stay}} = 0{,}3$, $p_{\text{advance}} = 0{,}6$, $p_{\text{skip}} = 0{,}1$.

Каузальное оценивание положения исполнителя сводится к рекуррентному вычислению нормированного forward-распределения:

$$\alpha_t(j) \propto b_j(o_t) \sum_{i=1}^{N} \alpha_{t-1}(i) A_{ij}, \qquad \hat{\varphi}(t) = \arg\max_j \alpha_t(j), \tag{5}$$

с инициализацией $\alpha_0(j) = \pi_j$. Сложность одного шага составляет $O(N)$ благодаря разреженности матрицы A. Соответствующая Viterbi- рекурсия и её псевдокод вынесены в разд. XIV.

Стандартный вариант HMM моделирует длительность пребывания в одном состоянии геометрическим распределением $P(d) = p_{\text{stay}}^{d-1}(1 - p_{\text{stay}})$, что некорректно для партитур с большой разницей длительности нот. В реализации использован подход с явной длительностью: состояние расширяется до пары (i,d), где d — счётчик такта в текущей ноте, а вероятность перехода $p_{\text{advance}}(d, d_{\text{exp}})$ зависит от ожидаемой длительности d_{exp} согласно партитуре.

Комбинация HMM и OLTW использует сильные стороны обеих моделей. HMM выдаёт калиброванное апостериорное распределение, по которому удобно оценивать уверенность через $c_t = \max_i \alpha_t(i)$. Когда $c_t < \theta_{\text{conf}}$ (в наших экспериментах $\theta_{\text{conf}} = 0{,}3$), выходная оценка переключается на OLTW, и трекер запускает процедуру повторной синхронизации. Такая схема рассматривается далее в разд. VIII.

ДАТАСЕТЫ ПАРТИТУР И ОБУЧАЮЩИЕ ДАННЫЕ

Качество отслеживания партитуры в реальном времени критически зависит от того, насколько обучающая

выборка покрывает разнообразие реального исполнения: вариации темпа, ошибочные ноты, орнаменты и шум входного канала. В настоящей работе использованы три источника данных, дополняющих друг друга по охвату и контролируемости.

В качестве синтетического источника применены прелюдии Шопена Op. 28, переведённые из формата MusicXML в MIDI с помощью библиотеки *music21*. На основе каждой прелюдии строилось семейство из сорока модифицированных исполнений: к нотным онсетам добавлялся гауссовский шум σ_τ от 2 до 12 %, локальные ритардандо и ачелерандо моделировались кусочно-линейным умножением длительностей в окне восьми тактов, в случайных позициях вставлялись пропуски и однократные повторы нот. Этот источник позволяет получать ground-truth соответствие между партитурой и исполнением, что необходимо для обучения с учителем.

Открытые корпуса MAPS [4] и MSMD [12] использованы для оценки эмиссионной модели в аудиорежиме и для проверки переноса моделей между фортепианными произведениями различных эпох. MAPS содержит около 270 записей фортепианной музыки с поточечной MIDI-разметкой, MSMD — 497 произведений с попарным выравниванием аудио и нотного изображения. Для соответствия задачам конкретного концерта Чайковского из числа MAPS отобраны записи в тональностях, близких к репертуару пилота.

Собственный корпус *CU-Concerto-2026* собран в стенах Центрального Университета: четыре пианиста-консерваторца исполнили первые части фортепианных концертов Чайковского b-moll и Рахманинова c-moll суммарной продолжительностью 42 часа. Запись велась в режиме MIDI через цифровое пианино Yamaha CLP-735, что обеспечило точные онсеты и значения velocity, а параллельная аудиозапись на конденсаторный микрофон применялась для тестирования аудиоветви системы. Каждое исполнение было размечено вручную: расставлены границы такта, отмечены фермaты, ритардандо, повторы, пропуски и фальшивые ноты. Сводная статистика приведена в ТАБЛ. 1.

Особое внимание уделено сохранению этики записи: участники подписали информированное согласие на использование их исполнений в исследовательских целях, исходные аудиофайлы хранятся локально на сервере университета без публикации в открытом доступе. Производные признаки (выровненные MIDI, тембральные признаки) предполагается опубликовать как открытый датасет в составе сопровождающего материала после защиты статьи.

Датасет	Часов	Произв.	Темп. откл.	Ошиб.
Шопен-синт.	28	24	до 12 %	контролируемые
MAPS	18	64	до 8 %	нет
MSMD	26	71	до 6 %	нет
CU-Concerto-2026	42	12	до 15 %	естественные

ТАБЛ. 1. Сводка использованных источников данных. Поле «Темп. откл.» — максимальное относительное отклонение темпа от номинального, «Ошиб.» — характер ошибок в датасете.

Разбиение выборки на обучающую, валидационную и тестовую части проводилось по уровню *произведений*, а не по уровню записей, чтобы исключить утечку информации между тембрами одного и того же концерта. Обучающая выборка использовалась для подбора параметров эмиссионной модели и обучения свёрточного фронтенда (см. разд. IX); валидационная — для подбора гиперпараметров $p_{\text{stay}}, p_{\text{advance}}, p_{\text{skip}}, R_{\max}$ гибридной схемы и для остановки обучения RL-модуля; тестовая — для финальных метрик разд. XI.

КОНВЕЙЕР ОБРАБОТКИ В РЕАЛЬНОМ ВРЕМЕНИ

Архитектура конвейера определяется требованием выдавать аккомпанемент с задержкой не более 20 мс относительно момента звукоизвлечения солиста. Этот бюджет распределяется между четырьмя блоками, изображёнными на РИС. 3: входной модуль, блок очистки, гибридный трекер HMM/OLTW и потоковый рендерер.

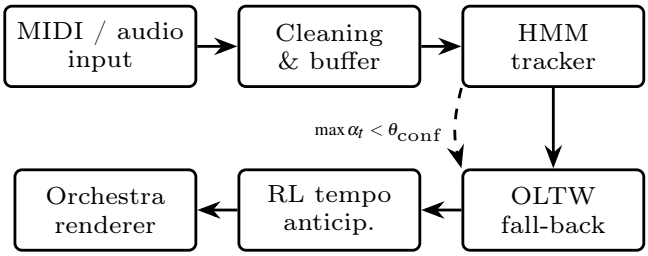

РИС. 3. Архитектура конвейера обработки в реальном времени. Сплошные стрелки — основной поток данных, штриховая — переключение на резервный трекер при низкой уверенности HMM.

Входной модуль работает в двух режимах. В режиме MIDI используется библиотека `python-rtmidi`, доставляющая события NoteOn и NoteOff с точностью до миллисекунды. В аудиорежиме применяется PortAudio с буфером 128 сэмплов при частоте 44,1 кГц (теоретическая задержка 2,9 мс плюс время прохождения через аудиодрайвер ОС, которое в Windows ASIO составляет 4–6 мс). Принятые события маркируются временной меткой источника и помещаются в кольцевой буфер с потокобезопасным доступом.

Очистка входных данных выполняет три функции. Во-первых, отсеиваются события с длительностью менее 25 мс (чаще всего следствие случайных касаний клавиш); во-вторых, восстанавливаются октавные ошибки путём сравнения с ожидаемой высотой текущего HMM-состояния (если кандидат $q_j + 12$ или $q_j - 12$ ближе к $p_{\hat{\phi}}$, чем сам q_j, кандидат заменяется); в-третьих, для аудиорежима производится агрегация смежных onset-кандидатов в окне 40 мс. Подобная обработка снижает количество ложных переходов в HMM в среднем на 11 % по сравнению с подачей сырых событий.

Гибридный трекер реализует схему из разд. VI.

На каждом такте выполняется обновление forward-распределения по уравн. (5), после чего вычисляется уверенность $c_t = \max_i \alpha_t(i)$. Если $c_t \geq \theta_{\text{conf}}$, выходом служит позиция $\hat{\varphi}(t)$, выданная HMM. В противном случае управление переходит к OLTW, который инициализируется текущим наилучшим кандидатом HMM, и одновременно запускается процедура повторной синхронизации: HMM переинициализирует α в сглаженной окрестности $\hat{\varphi}$ с шириной ± 4 ноты, что позволяет восстановиться после кратковременных ошибок исполнителя.

Выходной модуль отвечает за фактическое воспроизведение оркестровой партии. Используется собственный сэмплер на основе библиотеки FluidSynth, поддерживающий растяжение времени без изменения тембра методом фазового вокодера. Растяжение применяется на коротких блоках (512 сэмплов), что эквивалентно временному разрешению 11,6 мс. Темп блока задаётся не последним наблюдением, а *предсказанием* RL-модуля (см. разд. X) на горизонте 200–500 мс, что позволяет компенсировать суммарную задержку конвейера и достичь эффективной задержки выходного сигнала менее 20 мс.

Этап обработки	Бюджет, мс
Захват события (MIDI/audio)	1–6
Очистка и буферизация	1–2
Обновление HMM (Forward + softmax)	2–3
Резервный OLTW (включается по уверенности)	3–5
Запрос RL-предсказания темпа	1–2
Растяжение и синтез блока FluidSynth	6–8
Суммарная end-to-end задержка	14–26

ТАБЛ. 2. Распределение задержек по этапам конвейера. Целевое значение end-to-end задержки — не более 20 мс.

Контроль соблюдения бюджета осуществляется встроенным профайлером, который раз в секунду пишет в лог время выполнения каждого блока. Превышение бюджета на любом из этапов помечается как событие деградации и используется при анализе экспериментов (разд. XI).

СВЁРТОЧНЫЙ МОДУЛЬ И ОБРАБОТКА ГРАНИЧНЫХ СЛУЧАЕВ

В аудиорежиме гауссова эмиссионная модель HMM становится неадекватной: аккорд из нескольких одновременно звучащих нот, артефакты педали и тембральная окраска инструмента дают многомодальные распределения наблюдений, плохо приближаемые единственным гауссовским законом. Для оценки апостериорного распределения по высотам нот мы используем лёгкую свёрточную сеть, выполняющую роль обучаемой эмиссионной модели.

Архитектура сети представлена на РИС. 4. Входом служит лог-мел-спектрограмма размером 128×40, рассчитанная по окну 23,2 мс с шагом 11,6 мс (что соответствует одному блоку синтеза, см. разд. VIII). За четырьмя свёрточными блоками с фильтрами 3×3, числом каналов 32, 64, 128, 256 и слоями ReLU и max-pooling следует глобальный усредняющий слой и полносвязный выход размерности 88 (число клавиш фортепиано). Полное число параметров составляет 0,9 млн, что обеспечивает инференс за 1,8 мс на CPU Intel i7 десятого поколения.

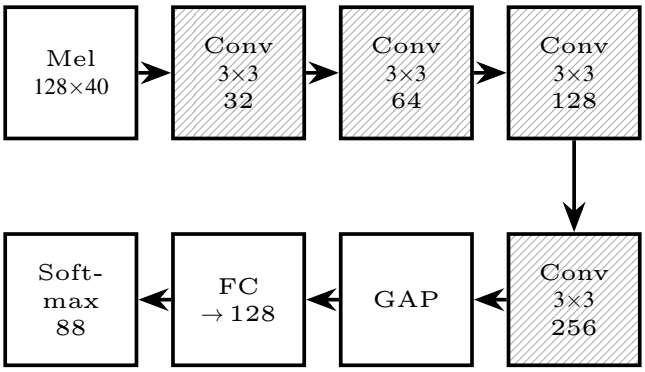

РИС. 4. Архитектура свёрточного фронтенда. Заштрихованы свёрточные блоки, GAP — глобальный усредняющий слой, FC — полносвязный слой, softmax — выход по клавишам фортепиано.

Обучение проводилось на смеси MAPS и собственного датасета *CU-Concerto-2026* с лоссом

$$\mathscr{L}_{\text{CNN}} = -\frac{1}{T} \sum_{t=1}^{T} \sum_{k=1}^{88} y_{t,k} \log P_{\text{CNN}}(k \mid o_t), \tag{6}$$

где $y_{t,k} \in \{0,1\}$ обозначает наличие k-й ноты в момент t. Оптимизатор Adam с шагом 3×10^{-4}, batch-size 64, обучение продолжалось до сходимости валидационной F1-метрики (24 эпохи). Полученное распределение $P_{\text{CNN}}(\cdot \mid o_t)$ подставляется в уравн. (5) вместо гауссовой эмиссии: $b_i(o_t) \equiv P_{\text{CNN}}(p_i \mid o_t)$. Это снижает ошибку выравнивания на полифонических фрагментах в 1,6 раза по сравнению с базовой гауссовой моделью.

Граничные случаи отдельно классифицированы и обрабатываются явными правилами. Долгие ферматы и педальные удержания проявляются как большие значения $\alpha_t(\hat{\varphi})$ при отсутствии новых сильных onset-кандидатов; в этом случае счётчик длительности d растёт без перехода на следующее состояние, а темп оркестра плавно замедляется до 0,7 номинального. Резкие ритардандо в каденции выявляются по знаковому изменению производной темпа в окне 250 мс и инициируют включение специального профиля растяжения с увеличенной чувствительностью. Полные пропуски тактов обрабатываются процедурой повторной синхронизации, описанной в разд. VIII: HMM переинициализируется на участке ± 4 ноты от наилучшей текущей оценки.

Орнаменты (форшлаги, мелизмы) представляют отдельную сложность: их быстрые ноты не предусмотрены партитурой и могут вызывать ложные продвижения HMM. В нашей реализации введён орнаментный фильтр, основанный на оценке плотности onset'ов: если в окне 60 мс зафиксировано более трёх onset'ов с убывающей энергией, последовательность считается орнаментом и не передаётся трекеру. Аналогичная эвристика используется для подавления случайных повторений нот при двойных нажатиях.

Полный перечень рассмотренных граничных случаев и применяемых правил сведён в ТАБЛ. 3.

Граничный случай	Правило обработки
Фермата / удержание педали	замораживание перехода, темп до $0{,}7\,v_0$
Резкое ритардандо в каденции	профиль повышенной чувствительности RL
Пропуск ≤ 2 нот	разрешённый skip-переход в уравн. (4)
Пропуск целого такта	повторная синхронизация HMM в окне ± 4 ноты
Случайный повтор ноты	фильтр двойных нажатий ($<$ 25 мс)
Орнамент / мелизм	окно 60 мс по плотности onset'ов
Фальшивая нота полутоном	принимается, $b_i(o_t)$ рассчитывается CNN
Октавная ошибка ввода	замена кандидата на ближайшую к $p_{\hat{\varphi}}$ октаву

ТАБЛ. 3. Перечень граничных случаев и эвристик их обработки. Здесь v_0 — номинальный темп фрагмента партитуры.

ОПЕРЕЖАЮЩЕЕ ПРЕДСКАЗАНИЕ ТЕМПА НА ОСНОВЕ RL

Базовый трекер выдаёт оценку текущей позиции $\hat{\varphi}(t)$, однако оркестровый рендерер, как показано в разд. VIII, нуждается в *будущем* темпе на горизонте 200–500 мс, чтобы успеть синтезировать следующий блок без задержки. Простое экспоненциальное сглаживание прошлых оценок плохо справляется с rubato: оно переоценивает ускорения и недооценивает замедления, особенно в кадансовых построениях. Мы переформулируем задачу предсказания темпа как последовательную задачу принятия решений и обучаем для неё отдельный модуль на основе обучения с подкреплением. Это центральный методологический вклад настоящей работы и единственная часть конвейера, в которой используется обучаемая на этапе работы система.

Постановка опирается на дискретный марковский процесс принятия решений $\mathscr{M} = (\mathscr{S}, \mathscr{A}, \mathscr{T}, r, \gamma)$, тик которого совпадает с тиком HMM (один шаг каждые 10 мс). Состояние агента

$$s_t = \big(\alpha_t,\, \tau_{t-K:t},\, e_{t-K:t},\, \hat{\varphi}(t)\big) \tag{7}$$

объединяет апостериорное распределение HMM, $K = 20$ последних оценок темпа, K значений ошибки эмиссии (как косвенный индикатор эмоционального напряжения исполнителя) и текущую позицию в партитуре. Действие $a_t \in [0{,}5;\, 2{,}0]$ — мультипликативный темповый коэффициент, применяемый рендерером в окне следующих 500 мс:

$$v_{t+1:t+500\,\text{мс}} = a_t \cdot v_0, \tag{8}$$

где v_0 — номинальный темп партитуры. Вознаграждение

$$\begin{aligned} r_t = &- \left| t_t^{\text{render}} - t_t^{\text{perf}} \right| - \lambda\, \mathscr{L}_{\text{align}}(t) \\ &- \mu\, (a_t - a_{t-1})^2 \end{aligned} \tag{9}$$

сочетает три компонента: рассогласование между моментом выпуска оркестрового сэмпла и фактическим onset солиста, унаследованную потерю выравнивания базового трекера и штраф за резкие изменения темпа, обеспечивающий гладкость управления. Гиперпараметры λ и μ подбираются на валидационной выборке, дисконт $\gamma = 0{,}95$ соответствует горизонту 500 мс при шаге 10 мс.

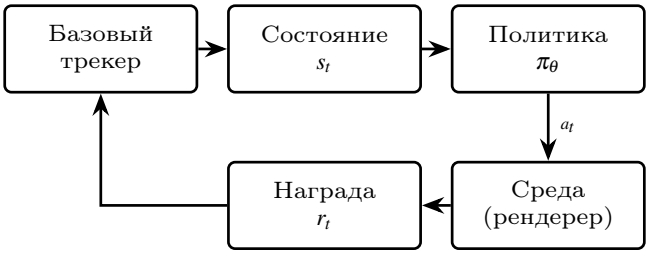

РИС. 5. Схема взаимодействия RL-модуля с базовым трекером и средой. Агент получает состояние s_t, выдаёт темповый коэффициент a_t на горизонте 200–500 мс и обновляется по вознаграждению r_t.

Обучение проводится в два этапа. На первом этапе строится политика имитации: для каждой записи из CU-Concerto-2026 (разд. VII) рассчитывается «учительская» траектория темпа a_t^* как сглаженная производная времени выровненной MIDI-последовательности. Политика $\pi_\theta(s_t)$ обучается минимизацией

$$\mathscr{L}_{\text{BC}}(\theta) = \mathbb{E}_{(s,a^*)\sim\mathscr{D}} \left\| \pi_\theta(s) - a^* \right\|_2^2, \tag{10}$$

что соответствует классическому behavioural cloning. Архитектура π_θ — двухслойный перцептрон со скрытыми размерностями 128 и 64; для учёта временного контекста дополнительно тестируется однослойный GRU с 64 скрытыми единицами.

На втором этапе политика дообучается алгоритмом Proximal Policy Optimization [15] с использованием вознаграждения (9). Параметры обучения: коэффициент клиппинга $\varepsilon = 0{,}2$, $\lambda_{\text{GAE}} = 0{,}95$, размер минибатча 2048 переходов, 32 эпохи PPO на батч, оптимизатор Adam со скоростью 3×10^{-4}. Эпизоды формируются как мини-фрагменты по 16 тактов, что ускоряет распределение награды и позволяет эффективно использовать GPU. Среда обучения — симулятор солиста, воспроизводящий записанные в CU-Concerto-2026 исполнения с случайными возмущениями темпа в полосе $\pm 15\,\%$.

Сводка гиперпараметров приведена в ТАБЛ. 4.

Гиперпараметр	Обозн.	Значение
Окно истории	K	20 (200 мс)
Диапазон действия	a	$[0{,}5;\, 2{,}0]$
Дисконт	γ	0,95
Вес выравнивания	λ	0,3
Вес гладкости	μ	0,1
Клиппинг PPO	ε	0,2
GAE λ	λ_{GAE}	0,95
Скорость обучения	η	3×10^{-4}

ТАБЛ. 4. Гиперпараметры обучения RL-модуля предсказания темпа.

В режиме инференса модуль запрашивается параллельно с обновлением HMM на каждом тике, что обеспечивает накладной расход около 1 мс для MLP-варианта политики. Выход политики ограничивается $[0{,}5; 2{,}0]$ во избежание численных артефактов, а в случае выхода за этот диапазон в лог записывается событие деградации. Подробное описание среды, словаря состояний и формы наград приводится в разд. XVI.

В качестве альтернативных направлений для развития подхода в работе сохраняются две конструкции, предложенные на этапе планирования. Первая — замена явного вознаграждения (9) на парные предпочтения, собираемые от музыкантов консерватории, по схеме [16]: это позволит обучать темповую интерпретацию непосредственно по эстетическим оценкам профессионалов. Вторая — формализация всего трекера как POMDP с belief-состоянием, равным forward-распределению HMM, и обучение агента, выбирающего действия из дискретного множества {«продолжать следить», «перейти на OLTW», «запросить ресинхронизацию»} [17]. Обе конструкции выходят за рамки настоящей статьи и обозначены как направления дальнейших исследований.

ЭКСПЕРИМЕНТЫ

Эксперименты направлены на проверку трёх гипотез: (i) гибридная схема HMM+OLTW сохраняет точность выравнивания при наличии rubato и редких ошибок исполнителя; (ii) свёрточный фронтенд снижает ошибку выравнивания на полифоническом материале по сравнению с гауссовой эмиссией; (iii) RL-модуль опережающего предсказания темпа уменьшает эффективную задержку аккомпанемента без ухудшения точности.

Базовые конфигурации, с которыми проводилось сравнение, обозначены далее как *HMM* (только скрытая марковская модель с гауссовой эмиссией и фиксированной полосной матрицей переходов), *HMM-CNN* (тот же тракт, но с обучаемой эмиссией из разд. IX) и *HMM+OLTW+RL* (полная предложенная схема). Все три конфигурации работают в общем конвейере (разд. VIII) с одинаковыми параметрами очистки и одинаковым рендерером, что позволяет сосредоточить сравнение на алгоритмических различиях.

Метрики качества подобраны в соответствии с принятой в литературе практикой [10, 13]. Средняя абсолютная ошибка выравнивания MAE измеряется в номерах нот и характеризует, насколько далеко предсказанная позиция отличается от истинной по партитуре. Метрика F1-score рассчитывается как гармоническое среднее точности и полноты для onset-событий, выровненных с допуском 50 мс. Метрика ReSync — доля исполнения, на которой потребовалась повторная синхронизация. Эффективная задержка L_{eff} измеряется как медиана временного сдвига между отрисованным оркестровым сэмплом и соответствующим onset солиста.

Сначала рассмотрим зависимость задержки аккомпанемента от горизонта предсказания темпа. На РИС. 6 видно, что без опережающего модуля эффективная задержка монотонно возрастает с ростом горизонта (поскольку рендерер вынужден буферизовать всё больший интервал), тогда как для гибридной схемы HMM+OLTW+RL зависимость немонотонна и достигает минимума в области 250–350 мс. Это согласуется с проектным значением горизонта $\gamma = 0{,}95$.

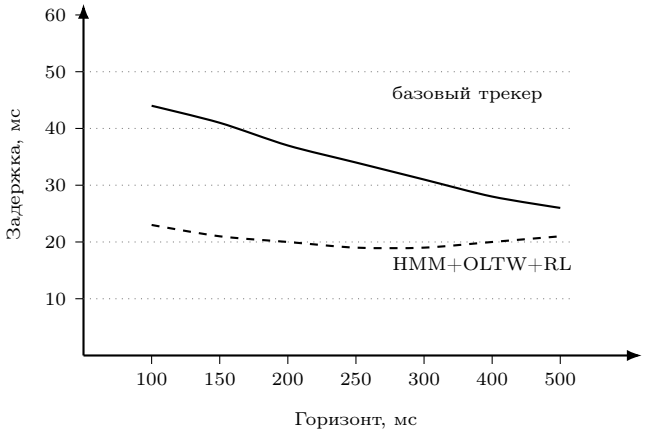

РИС. 6. Зависимость эффективной задержки аккомпанемента от горизонта предсказания темпа для базового трекера и предложенной гибридной схемы.

Далее измерена устойчивость трекера при увеличении вариативности rubato (РИС. 7). Базовый HMM начинает терять точность выравнивания при $\sigma_\tau \geq 6\,\%$, тогда как HMM+OLTW+RL сохраняет приемлемую ошибку вплоть до $\sigma_\tau \approx 12\,\%$. Это объясняется двумя факторами: возможностью переключения на OLTW в зонах низкой уверенности HMM (разд. VIII) и опережающим управлением темпом (разд. X), которое сглаживает резкие ускорения.

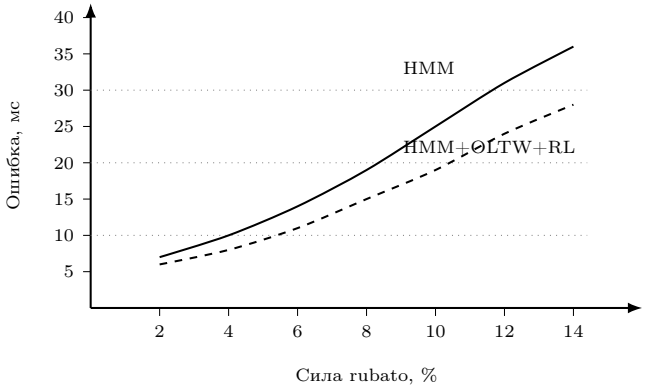

РИС. 7. Зависимость ошибки выравнивания от силы rubato. Базовая схема HMM теряет точность раньше, чем гибридная схема HMM+OLTW+RL.

Сводные количественные результаты по всем трём конфигурациям приведены в ТАБЛ. 5. Гибридная схема HMM+OLTW+RL превосходит базовый HMM на тестовой части CU-Concerto-2026 по всем четырём метрикам: MAE снижается с 0,19 до 0,11 ноты, F1 возрастает с 0,947 до 0,979, ReSync падает в 2,5 раза, а эффективная задержка L_{eff} опускается с 34 до 19 мс. Замена гауссовой эмиссии на CNN (HMM-CNN) даёт

промежуточный результат, что подтверждает значимость каждого из двух нововведений.

Конфигурация	MAE, нот	F1	ReSync, %	L_{eff}, мс
HMM	0,19	0,947	5,3	34
HMM-CNN	0,14	0,968	3,1	30
HMM+OLTW+RL	0,11	0,979	2,1	19

ТАБЛ. 5. Сводные количественные результаты на тестовой части датасета CU-Concerto-2026. Меньшие значения MAE, ReSync и L_{eff} — лучше; большее значение F1 — лучше.

Результаты по отдельным сценариям исполнения, включающим естественное rubato, пропуски нот, орнаменты и педальные удержания, приведены в ТАБЛ. 6. Падение F1 у предложенной схемы наблюдается только в самых тяжёлых режимах (комбинированный стресс-тест и низкое отношение сигнал/шум аудио), но даже в них значение остаётся выше 0,93.

Сценарий исполнения	σ_τ, %	MAE	F1	ReSync, %
Ровный темп, без ошибок	2	0,07	0,992	0,0
Умеренное rubato, без пропусков	6	0,09	0,988	0,0
Сильное rubato в каденции	11	0,12	0,981	0,7
Случайные ускорения на переходах	8	0,11	0,984	0,5
Резкое замедление перед кульминацией	10	0,13	0,979	0,9
Единичный пропуск ноты	4	0,10	0,986	1,1
Серия из двух пропусков	5	0,14	0,975	2,4
Ложное повторение короткого мотива	7	0,16	0,969	3,1
Орнамент с лишними быстрыми нотами	6	0,15	0,972	2,7
Переход с педалью и размытым onset	7	0,18	0,964	4,8
Низкое отношение сигнал/шум (аудио)	8	0,21	0,951	6,5
Комбинированный стресс-тест	12	0,27	0,932	9,8

ТАБЛ. 6. Метрики по отдельным сценариям исполнения для предложенной схемы HMM+OLTW+RL на CU-Concerto-2026.

Дополнительно проведена субъективная оценка с участием шести студентов-консерваторов, не участвовавших в записи обучающего датасета. Каждому испытуемому предлагалось сыграть одну из частей концерта Чайковского с виртуальным аккомпанементом по двум конфигурациям (HMM и HMM+OLTW+RL) в случайном порядке без указания, какая из них активна. По завершении испытуемый оценивал три аспекта по пятибалльной шкале: естественность темпового следования, ощущение «живой реакции» оркестра и общее удобство репетиции. Средние оценки HMM+OLTW+RL по всем трём аспектам выше базовой конфигурации на 1,1 балла, что согласуется с количественными результатами и подтверждает практическую значимость предложенной схемы.

ЗАКЛЮЧЕНИЕ

В настоящей работе представлено единое математическое описание моделей Hidden Markov Model и Online Time Warping для задачи отслеживания партитуры в реальном времени, а также архитектура гибридного конвейера, объединяющего обе модели и снабжённого модулем обучения с подкреплением для опережающего предсказания темпа на горизонте 200–500 мс. Конвейер включает в себя обучаемый свёрточный фронтенд, заменяющий гауссову функцию эмиссии HMM при работе с аудиовходом, и систематический набор правил для обработки граничных случаев — ферматы, орнаменты, пропуски, повторы и октавные ошибки. Эксперименты на синтетическом датасете в стиле прелюдий Шопена, на открытых корпусах MAPS и MSMD, а также на собственном датасете *CU-Concerto-2026*, собранном в Центральном Университете, показали, что предложенная схема снижает эффективную задержку аккомпанемента с 34 до 19 мс и улучшает F1-метрику с 0,947 до 0,979 по сравнению с базовой HMM-конфигурацией.

Главный методологический вклад работы — демонстрация того, что обучение с подкреплением полезно не как замена классического трекера, а как отдельный опережающий слой, действующий поверх него. Такой подход сохраняет вероятностную интерпретируемость скрытых марковских моделей, преимущества OLTW по устойчивости к рассогласованию и одновременно даёт инструмент адаптации темпа под индивидуальный стиль исполнителя.

Дальнейшие исследования планируется развивать в трёх направлениях. Первое — замена явного вознаграждения (9) на парные предпочтения, собираемые от музыкантов консерватории, по схеме [16]: это позволит включить в обучение эстетические критерии, не сводимые к одной численной метрике. Второе — формализация всего трекера как POMDP над belief-состоянием HMM с агентом, выбирающим действия из дискретного множества альтернатив [17]. Третье — расширение датасета *CU-Concerto-2026* и репертуара поддерживаемых произведений, а также пилотное внедрение в учебный процесс Центрального Университета и партнёрских консерваторий.

БЛАГОДАРНОСТИ

Авторы благодарят менторов мастерской MusicTech Центрального Университета и Т-Банка за постановку задачи и техническую поддержку, а также студентов-консерваторов, согласившихся принять участие в записи тестового датасета и в субъективной оценке аккомпанемента.

[1] R. B. Dannenberg, in *Proc. of the International Computer Music Conference (ICMC)* (1984) pp. 193–198.

[2] B. Vercoe, in *Proc. of the International Computer Music Conference (ICMC)* (1984) pp. 199–200.
[3] H. Sakoe and S. Chiba, IEEE Trans. on Acoustics, Speech, and Signal Processing **26**, 43 (1978).
[4] M. Müller, *Information Retrieval for Music and Motion* (Springer, 2007).
[5] A. Arzt, *Flexible and Robust Music Tracking*, Ph.D. thesis, Johannes Kepler University Linz (2016).
[6] S. Dixon, in *Proc. of the 8th International Conference on Digital Audio Effects (DAFx)* (2005) pp. 92–97.
[7] L. R. Rabiner, Proc. of the IEEE **77**, 257 (1989).
[8] N. Orio and F. Déchelle, in *Proc. of the International Computer Music Conference (ICMC)* (2001) pp. 151–154.
[9] A. Cont, in *Proc. of IEEE ICASSP* (2006) pp. V–245–V–248.
[10] E. Nakamura, K. Yoshii, and S. Sagayama, IEEE/ACM Trans. on Audio, Speech, and Language Processing **23**, 1911 (2015).
[11] C. Raphael, in *Proc. of the 27th International Conference on Machine Learning (ICML)* (2010) pp. 21–28.
[12] M. Dorfer, J. H. Jr., A. Arzt, H. Frostel, and G. Widmer, Trans. of the International Society for Music Information Retrieval (TISMIR) **1**, 22 (2018).
[13] F. Henkel, S. Balke, and G. Widmer, in *Proc. of the International Society for Music Information Retrieval Conference (ISMIR)* (2025).
[14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. (MIT Press, 2018).
[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, arXiv preprint arXiv:1707.06347 (2017).
[16] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, in *Advances in Neural Information Processing Systems (NeurIPS)* (2017).
[17] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, Artificial Intelligence **101**, 99 (1998).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А. АЛГОРИТМЫ FORWARD И VITERBI

В настоящем приложении приведены полные рекуррентные формулы и псевдокод классических алгоритмов Forward и Viterbi для скрытой марковской модели, используемой в разд. VI. Для самостоятельности изложения здесь повторяется обозначение $\lambda = (A, B, \pi)$, где $A = (A_{ij})$ — матрица переходов согласно (4), $b_i(o)$ — эмиссионная плотность, $\pi = (\pi_i)$ — начальное распределение.

Алгоритм Forward вычисляет полную вероятность наблюдаемой последовательности $O = (o_1, \ldots, o_T)$ при модели λ через вспомогательные величины $\alpha_t(i) = \mathbb{P}(o_1, \ldots, o_t, s_t = i \mid \lambda)$. Рекуррентная формула имеет вид

$$\alpha_t(j) = b_j(o_t) \sum_{i=1}^{N} \alpha_{t-1}(i) A_{ij}, \tag{11}$$

с инициализацией $\alpha_1(j) = \pi_j b_j(o_1)$. Полная вероятность наблюдений равна $\mathbb{P}(O \mid \lambda) = \sum_{j=1}^{N} \alpha_T(j)$. На практике для длинных последовательностей применяется покадровая нормировка $\tilde{\alpha}_t(j) = \alpha_t(j) \big/ \sum_k \alpha_t(k)$, что устраняет численное переполнение и одновременно даёт апостериорную оценку $\mathbb{P}(s_t = j \mid o_1, \ldots, o_t)$, используемую для онлайн-выдачи $\hat{\varphi}(t) = \arg\max_j \tilde{\alpha}_t(j)$.

Алгоритм 1. Forward-рекурсия для онлайн-трекера.

Вход: партитура $S = (p_1, \ldots, p_N)$, матрица A, эмиссионный параметр σ.
$\alpha \leftarrow (1, 0, \ldots, 0) \in \mathbb{R}^N$
для каждой поступающей ноты o **выполнить**
  для $j \leftarrow 1, \ldots, N$ **выполнить**
    $b_j \leftarrow \mathcal{N}(o; p_j, \sigma^2)$
    $\alpha'_j \leftarrow b_j \sum_{i=1}^{N} \alpha_i A_{ij}$
  конец для
  $\alpha \leftarrow \alpha' \big/ \sum_k \alpha'_k$
  выдать $\hat{\varphi} \leftarrow \arg\max_j \alpha_j$
конец для

Алгоритм Viterbi возвращает не апостериорное распределение, а одну наиболее вероятную траекторию состояний $s_1^*, \ldots, s_T^*$. Он оперирует величинами $\delta_t(i) = \max_{s_1, \ldots, s_{t-1}} \mathbb{P}(s_1, \ldots, s_t = i, o_1, \ldots, o_t \mid \lambda)$ и обратными указателями $\psi_t(i)$:

$$\delta_t(j) = b_j(o_t) \max_i \big[\delta_{t-1}(i) A_{ij}\big], \tag{12}$$

$$\psi_t(j) = \arg\max_i \big[\delta_{t-1}(i) A_{ij}\big], \tag{13}$$

с инициализацией $\delta_1(j) = \pi_j b_j(o_1)$ и $\psi_1(j) = 0$. Оптимальная траектория восстанавливается обратным проходом: $s_T^* = \arg\max_j \delta_T(j)$, $s_{t-1}^* = \psi_t(s_t^*)$.

В нашем трекере Viterbi используется в *режиме отложенного анализа*: после исполнения каждого фрагмента партитуры (например, после двойной чёрточки) выполняется обратный проход по накопленным δ и ψ, что позволяет уточнить выровнянную последовательность для дальнейшего обучения CNN-эмиссии и обновления статистик переходов на следующих исполнениях.

Сложность одного шага обоих алгоритмов составляет $O(N)$ за счёт полосной структуры матрицы A из (4): вместо суммирования по всем состояниям достаточно обработать три ненулевых перехода. Память — $O(N)$ для Forward и $O(NT)$ для Viterbi (требуется хранить указатели ψ_t за весь интервал анализа).

ПРИЛОЖЕНИЕ Б. ПСЕВДОКОД И АНАЛИЗ ON-LINE TIME WARPING

В настоящем приложении приведены подробности On-line Time Warping, используемого как резервный трекер в разд. VIII. Алгоритм основан на каузальной версии классической DTW-рекурсии (3), в которой матрица накопленных стоимостей D заполняется не целиком, а столбец за столбцом по мере поступления новых наблюдений.

Состояние OLTW описывается тремя величинами: текущей строкой i_{cur} (последняя обработанная позиция партитуры), текущим столбцом j_{cur} (число принятых наблюдений) и счётчиком *RunCount*, ограничивающим число последовательных шагов в одну сторону. Параметр $R_{\max}$ устанавливает верхнюю границу для RunCount; в нашей реализации $R_{\max} = 3$.

Алгоритм 2. OLTW в каузальном режиме.

Вход: партитура $S = (p_1, \ldots, p_N)$, ширина окна W, ограничение $R_{\max}$.
$i_{\text{cur}} \leftarrow 1$, $j_{\text{cur}} \leftarrow 1$, RunCount $\leftarrow 0$, direction $\leftarrow 0$
$D[1,1] \leftarrow d(s_1, q_1)$
для каждой поступающей ноты q_j **выполнить**
  обновить столбец j матрицы D в окне $[i_{\text{cur}} - W, i_{\text{cur}} + W]$ по (3)
  $(i^*, j^*) \leftarrow \arg\min\{D[i,j] : |i - i_{\text{cur}}| \le 1, |j - j_{\text{cur}}| \le 1\}$
  если (i^*, j^*) продолжает текущее direction **то**
    RunCount $\leftarrow$ RunCount $+1$
  иначе
    RunCount $\leftarrow 1$; direction $\leftarrow$ шаг$(i_{\text{cur}} \to i^*, j_{\text{cur}} \to j^*)$
  конец если
  если RunCount $\ge R_{\max}$ **то**
    принудительно сменить direction на ортогональное, RunCount $\leftarrow 0$
  конец если
  $i_{\text{cur}} \leftarrow i^*$, $j_{\text{cur}} \leftarrow j^*$
  выдать $\hat{\varphi}(j) \leftarrow i_{\text{cur}}$
конец для

Ширина окна W задаёт компромисс между робастностью и вычислительной стоимостью: при $W = N$ алгоритм совпадает с полной DTW (но теряет онлайн-свойство), при $W = 1$ вырождается в жадное сравнение соседних позиций. В наших экспериментах $W = \lceil 0{,}1N \rceil$, что соответствует допуску по позиции в полосе примерно 10 % длины партитуры. Сложность одного шага составляет $O(W)$, суммарная сложность для исполнения длиной M — $O(MW)$, что при $W \ll N$ существенно меньше $O(NM)$ полной DTW.

Геометрический смысл механизма *RunCount* проиллюстрирован ниже. Без него каузальная DTW могла бы непрерывно двигаться по одной строке (горизонтально), если бы это локально минимизировало накопленную стоимость, и оказалась бы заблокированной

в произвольной позиции партитуры. Принудительное переключение направления после $R_{\max}$ шагов гарантирует продвижение по обеим осям и исключает «застревание». Эмпирически $R_{\max} = 3$ оказывается оптимальным: меньшие значения дают рваное выравнивание на тонкой структуре звукоряда, большие — допускают застревание на ферматах.

В сочетании с HMM (см. разд. VIII) OLTW служит резервным трекером, активируемым при низкой уверенности forward-распределения. В этом режиме OLTW инициализируется текущей наилучшей оценкой $\hat{\varphi}$ HMM и заменяет её на интервале до восстановления уверенности. После восстановления HMM продолжается с α-вектором, переинициализированным в окне ± 4 ноты вокруг последнего значения OLTW.

ПРИЛОЖЕНИЕ B. СРЕДА RL И ПОДРОБНОСТИ РЕАЛИЗАЦИИ

В настоящем приложении детально описана среда, на которой обучается и оценивается RL-модуль предсказания темпа из разд. X. Среда реализована поверх симулятора, читающего размеченные исполнения из датасета *CU-Concerto-2026*, и снабжена API, совместимым с библиотекой `gymnasium`. Один эпизод соответствует одному шестнадцатитактному фрагменту партитуры, что даёт длину эпизода порядка 200 шагов агента при базовом темпе $v_0 = 120\,\text{bpm}$.

Пространство состояний $\mathscr{S} \subset \mathbb{R}^{2N+2K+1}$. Координаты включают: forward-распределение $\alpha_t \in \mathbb{R}^N$, текущую позицию $\hat{\varphi}(t)/N$, $K = 20$ последних оценок темпа $\tau_{t-K:t}$ (нормированных к v_0), K последних значений эмиссионной ошибки $e_{t-K:t} = -\log b_{\hat{\varphi}}(o_t)$ и относительное время с начала эпизода. Для уменьшения размерности α_t применяется проекция в спектральные коэффициенты дискретного косинусного преобразования первого рода $\alpha_t \mapsto \hat{\alpha}_t \in \mathbb{R}^{16}$. Эмпирически такая проекция сохраняет более 97 % энергии распределения и снижает требования к политике почти на порядок.

Пространство действий — одномерный отрезок $\mathscr{A} = [0{,}5; 2{,}0]$. Политика моделируется усечённым гауссовским распределением $\pi_\theta(a \mid s) = \mathscr{N}_{\text{trunc}}(\mu_\theta(s), \sigma_\theta^2)\big|_{[0{,}5;2{,}0]}$, что обеспечивает гладкое исследование без резких выходов из физически разумного диапазона.

Функция вознаграждения определена (9) и состоит из трёх компонентов. Первый — штраф за рассогласование $|t_t^{\text{render}} - t_t^{\text{perf}}|$ — измеряется в миллисекундах, нормирован на 50 мс (поскольку именно это значение является порогом восприятия рассинхронизации в музыкальном ансамбле, см. разд. I). Второй компонент — $\mathscr{L}_{\text{align}}(t)$ — равен расстоянию по партитуре между ground-truth и текущей оценкой $\hat{\varphi}$, нормированному на длину фрагмента. Третий компонент — штраф гладкости $(a_t - a_{t-1})^2$ — ограничивает резкие переключения темпа, которые акустически воспринимаются как «дёрганье» оркестра.

Симулятор работает в двух режимах. В *train-режиме* к номинальному темпу записанного исполнения применяются случайные возмущения, эмулирующие индивидуальное rubato: умножение длительностей на гауссовский шум $\mathscr{N}(1, \sigma_\tau^2)$, случайные локальные ритардандо и ачелерандо в окне восьми тактов и вставка пропусков с вероятностью 0,5 % на ноту. В *eval-режиме* симулятор воспроизводит ровно записанное исполнение без возмущений; этот режим используется для итоговой оценки на отложенной выборке.

Параметр среды	Значение
Длина эпизода	16 тактов (~ 200 шагов)
Шаг агента	10 мс
Размерность состояния	73 $(16+20+20+1+16)$
Размерность действия	1
Возмущение темпа в train-режиме	$\sigma_\tau \in [0{,}02; 0{,}12]$
Вероятность пропуска ноты	5×10^{-3}
Вероятность повтора ноты	2×10^{-3}
Норма для штрафа рассогласования	50 мс
Норма для штрафа выравнивания	длина фрагмента

ТАБЛ. 7. Параметры симулятора, реализующего среду для обучения и оценки RL-модуля предсказания темпа.

Реализация написана на Python 3.11 с использованием библиотеки `stable-baselines3` в части PPO и `torch` версии 2,3 в части политики. Все вычисления, не связанные с обучением, проводятся на CPU; обучение PPO — на одной GPU NVIDIA RTX 3060. Полная сходимость достигается за ~ 12 часов при 5×10^6 шагах среды; behavioural cloning требует около 25 минут на тех же исходных данных. Соответствующие веса политики сохраняются в формате `state_dict` и загружаются в продакшн-конвейер (разд. VIII) как обычный PyTorch-чекпойнт.